

What is JavaScript?

- **JavaScript** is a **high-level, interpreted programming language** used to make web pages **interactive** and **dynamic**.
- It runs directly in the **browser** and can also run on the **server** using **Node.js**.
- It can:
 - Add interactivity (like buttons, animations, forms).
 - Manipulate HTML and CSS.
 - Handle user input and events.
 - Communicate with servers (using APIs).

Example:

```
alert("Hello, JavaScript!");
```

Variables in JavaScript

Variables are used to **store data values**.
There are **three ways** to declare a variable:

Keyword	Reassignment	Redeclaration	Example
var	Yes	Yes	var x = 10;
let	Yes	No	let y = 20;
const	No	No	const z = 30;

Naming Rules for Variables:

- ✓ Variable names are **case-sensitive** (age ≠ Age).
- ✓ Must **start with** a letter, **_**, or **\$** (not a number).
- ✓ Can **contain letters, digits, underscores, and \$**.
- ✓ Cannot use **JavaScript reserved keywords** (like let, if, for, etc.).
- ✓ Use **camelCase** naming convention for readability.

Example:

```
let firstName = "John";  
const PI = 3.14;  
var age = 25;
```

JavaScript Data Types

JavaScript has **7 primitive data types**:

Type	Description	Example
Number	Numeric values	let x = 10;
String	Text inside quotes	let name = "Neeraj";

Type	Description	Example
Boolean	True or false values	let isLoggedIn = true;
Null	Intentional empty value	let car = null;
Undefined	Declared but no value assigned	let value;
Symbol	Unique and immutable value	let id = Symbol("id");
BigInt	Very large integers	let bigNum = 12345678901234567890n;

Type Conversion

Type conversion means **changing data from one type to another**.

A. Implicit Conversion (Type Coercion)

JavaScript automatically converts data types.

Example:

```
let result = "5" + 2; // "52" (number converted to string)
let sum = "5" - 2;    // 3   (string converted to number)
```

B. Explicit Conversion

Manually converting types using functions.

Conversion	Method	Example	Result
To Number	Number()	Number("10")	10
To String	String()	String(100)	"100"
To Boolean	Boolean()	Boolean(0)	false

JavaScript Operators

Operators are **symbols** used to perform **operations on values and variables**.

A. Arithmetic Operators

Used to perform **mathematical calculations**.

Operator	Description	Example	Output
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	10 / 2	5
%	Modulus (Remainder)	10 % 3	1

Operator	Description	Example	Output
**	Exponentiation	2 ** 3	8
++	Increment	let x = 5; x++;	6
--	Decrement	let x = 5; x--;	4

Example Code:

```
let a = 10;
let b = 3;

console.log(a + b); // 13
console.log(a - b); // 7
console.log(a * b); // 30
console.log(a / b); // 3.333...
console.log(a % b); // 1
console.log(a ** b); // 1000
```

B. Assignment Operators

Used to **assign values** to variables.

Operator	Description	Example	Same as
=	Assign	x = 5	x = 5
+=	Add and assign	x += 3	x = x + 3
-=	Subtract and assign	x -= 2	x = x - 2
*=	Multiply and assign	x *= 4	x = x * 4
/=	Divide and assign	x /= 2	x = x / 2
%=	Modulus and assign	x %= 3	x = x % 3

Example Code:

```
let x = 10;
x += 5; // x = 15
x -= 3; // x = 12
x *= 2; // x = 24
x /= 4; // x = 6
console.log(x);
```

C. Comparison Operators

Used to **compare two values** and return a **Boolean** (true or false).

Operator	Description	Example	Result
==	Equal to (compares value only)	5 == "5"	true
===	Strict equal (compares value + type)	5 === "5"	false
!=	Not equal	5 != 8	true
!==	Strict not equal	5 !== "5"	true

Operator	Description	Example	Result
>	Greater than	10 > 5	true
<	Less than	10 < 5	false
>=	Greater than or equal to	10 >= 10	true
<=	Less than or equal to	5 <= 8	true

Example Code:

```
let a = 10, b = "10";

console.log(a == b); // true
console.log(a === b); // false
console.log(a != b); // false
console.log(a !== b); // true
console.log(a > 5); // true
console.log(a <= 10); // true
```

D. Logical Operators

Used to **combine multiple conditions**.

Operator	Description	Example	Result
&&	AND	(5 > 2 && 3 < 5)	true
	OR	(5 > 2 3 < 5)	true
!	NOT	!(5 > 3)	false

Example Code:

```
let age = 20;
let hasID = true;

if (age >= 18 && hasID) {
  console.log("Allowed to vote");
}

let isStudent = false;
if (isStudent || age < 18) {
  console.log("Discount available");
}

console.log(!true); // false
```

E. Ternary Operator

A **short form of if-else** statement.

Syntax:

```
condition ? valueIfTrue : valueIfFalse
```

Example Code:

```
let age = 18;
let message = (age >= 18) ? "Adult" : "Minor";
console.log(message); // Output: Adult
```

Conditional Statements (If-Else)

The `if...else` statement is used to **make decisions** in a program based on **conditions**. It checks a condition and executes code **only if** that condition is true.

A. Basic `if` Statement

Syntax:

```
if (condition) {
  // code to run if condition is true
}
```

Example:

```
let age = 18;

if (age >= 18) {
  console.log("You are eligible to vote.");
}
```

Output:

You are eligible to vote.

B. `if...else` Statement

Used when we want to run **one block of code if the condition is true**, and another block if it's false.

Syntax:

```
if (condition) {
  // code if condition is true
} else {
  // code if condition is false
}
```

Example:

```
let marks = 45;

if (marks >= 50) {
  console.log("You passed the exam!");
} else {
  console.log("You failed the exam.");
}
```

Output:

You failed the exam.

C. if...else if...else Statement

Used when there are **multiple conditions** to check.

Syntax:

```
if (condition1) {  
    // code if condition1 is true  
} else if (condition2) {  
    // code if condition2 is true  
} else {  
    // code if both are false  
}
```

Example:

```
let score = 85;  
  
if (score >= 90) {  
    console.log("Grade: A");  
} else if (score >= 75) {  
    console.log("Grade: B");  
} else if (score >= 50) {  
    console.log("Grade: C");  
} else {  
    console.log("Grade: Fail");  
}
```

Output:

Grade: B

D. Nested if Statement

An if statement **inside another if** is called **nested if**.

Example:

```
let username = "admin";  
let password = "1234";  
  
if (username === "admin") {  
    if (password === "1234") {  
        console.log("Login successful!");  
    } else {  
        console.log("Incorrect password!");  
    }  
} else {  
    console.log("Invalid username!");  
}
```

Output:

Login successful!

switch-case Statement

The `switch` statement is used to **compare one expression with multiple values** (called *cases*).

It is an alternative to writing many `if...else if` statements.

Syntax:

```
switch (expression) {  
  case value1:  
    // code block  
    break;  
  
  case value2:  
    // code block  
    break;  
  
  default:  
    // code block if no match found  
}
```

Example 1: Day of the Week

```
let day = 3;  
let dayName;  
  
switch (day) {  
  case 1:  
    dayName = "Monday";  
    break;  
  case 2:  
    dayName = "Tuesday";  
    break;  
  case 3:  
    dayName = "Wednesday";  
    break;  
  case 4:  
    dayName = "Thursday";  
    break;  
  case 5:  
    dayName = "Friday";  
    break;  
  case 6:  
    dayName = "Saturday";  
    break;  
  case 7:  
    dayName = "Sunday";  
    break;  
  default:  
    dayName = "Invalid day";  
}  
  
console.log(dayName);
```

Output:

Wednesday

Example 2: Simple Calculator

```
let a = 10;
let b = 5;
let operator = "*";
let result;

switch (operator) {
  case "+":
    result = a + b;
    break;
  case "-":
    result = a - b;
    break;
  case "*":
    result = a * b;
    break;
  case "/":
    result = a / b;
    break;
  default:
    result = "Invalid operator";
}

console.log("Result:", result);
```

Output:

Result: 50

8. Loops in JavaScript

Definition:

A **loop** is used to **repeat a block of code multiple times** until a condition becomes false.

There are different types of loops in JavaScript:

1. for loop
2. while loop
3. do...while loop
4. for...of loop
5. for...in loop

A. for Loop

Used when the number of repetitions is **known** in advance.

Syntax:

```
for (initialization; condition; update) {
  // code to execute
}
```


Example:

```
for (let i = 1; i <= 5; i++) {  
  console.log("Number:", i);  
}
```

Output:

```
Number: 1  
Number: 2  
Number: 3  
Number: 4  
Number: 5
```

B. while Loop

Used when the number of iterations is **not known** in advance.
The loop runs **while the condition is true**.

Syntax:

```
while (condition) {  
  // code to execute  
}
```

Example:

```
let i = 1;  
while (i <= 5) {  
  console.log("Count:", i);  
  i++;  
}
```

Output:

```
Count: 1  
Count: 2  
Count: 3  
Count: 4  
Count: 5
```

C. do...while Loop

The **do...while** loop runs the code **at least once**, even if the condition is false.

Syntax:

```
do {  
  // code to execute  
} while (condition);
```

Example:

```
let i = 1;
```

```
do {
  console.log("Value:", i);
  i++;
} while (i <= 3);
```

Output:

```
Value: 1
Value: 2
Value: 3
```

Example (Condition False Initially):

```
let x = 10;
do {
  console.log("Runs at least once!");
} while (x < 5);
```

Output:

```
Runs at least once!
```

9. break and continue Statements

Both are **loop control statements** used to change how loops work.

A. break Statement

- The `break` statement **stops** the loop **immediately** — even if the condition is still true.
- Control jumps out of the loop.

Syntax:

```
break;
```

Example:

```
for (let i = 1; i <= 10; i++) {
  if (i === 5) {
    break; // Stop loop when i = 5
  }
  console.log(i);
}
```

Output:

```
1
2
3
4
```

Explanation: The loop stops completely when `i` becomes 5.

B. continue Statement

- The `continue` statement **skips the current iteration** and moves to the **next iteration** of the loop.

Syntax:

```
continue;
```

Example:

```
for (let i = 1; i <= 5; i++) {  
  if (i === 3) {  
    continue; // Skip 3  
  }  
  console.log(i);  
}
```

Output:

```
1  
2  
4  
5
```

Explanation: When `i = 3`, the loop skips printing and continues with the next value.

Example with while loop

```
let i = 0;  
  
while (i < 5) {  
  i++;  
  if (i === 2) continue;  
  if (i === 4) break;  
  console.log(i);  
}
```

Output:

```
1  
3
```

10. Functions in JavaScript

A **function** is a **block of reusable code** that performs a specific task. It helps make code **modular, readable, and reusable**.

A. Function Declaration

Syntax:

```
function functionName(parameters) {
```

```
    // code to execute
}
```

Example:

```
function greet() {
    console.log("Hello, JavaScript!");
}

greet(); // Function call
```

Output:

```
Hello, JavaScript!
```

B. Function with Parameters and Arguments

Parameters act like variables that accept values when a function is called.

Example:

```
function add(a, b) {
    console.log("Sum:", a + b);
}

add(5, 3); // Arguments
```

Output:

```
Sum: 8
```

C. Function with Return Value

Functions can **return a result** using the `return` keyword.

Example:

```
function multiply(x, y) {
    return x * y;
}

let result = multiply(4, 5);
console.log("Result:", result);
```

Output:

```
Result: 20
```

D. Function Expression

A function can be stored in a **variable** — called a **function expression**.

Example:

```
const subtract = function(a, b) {  
  return a - b;  
};  
  
console.log(subtract(10, 4));
```

Output:

6

E. Arrow Functions (ES6)

A shorter way to write functions using the => arrow symbol.

Syntax:

```
const functionName = (parameters) => {  
  // code  
};
```

Example 1:

```
const greet = () => {  
  console.log("Welcome!");  
};  
  
greet();
```

Output:

Welcome!

Arrays in JavaScript

An **array** is a **collection of values** stored in a single variable. Each value is stored at a numeric **index**, starting from **0**.

A. Array Creation

1. Using square brackets []:

```
let fruits = ["Apple", "Banana", "Cherry"];  
console.log(fruits);
```

Output:

```
["Apple", "Banana", "Cherry"]
```

2. Using the Array constructor:

```
let numbers = new Array(10, 20, 30);  
console.log(numbers);
```

Output:

```
[10, 20, 30]
```

B. Array Indexing

- Index starts from 0
- You can access or change elements using **indexes**

Example:

```
let colors = ["Red", "Green", "Blue"];
console.log(colors[0]); // Red
console.log(colors[2]); // Blue

colors[1] = "Yellow"; // Update element
console.log(colors);
```

Output:

```
Red
Blue
["Red", "Yellow", "Blue"]
```

Array Methods (Basic)

A. push()

Adds an element **at the end** of the array.

```
let fruits = ["Apple", "Banana"];
fruits.push("Cherry");
console.log(fruits);
```

Output:

```
["Apple", "Banana", "Cherry"]
```

B. pop()

Removes the **last element** from the array.

```
let fruits = ["Apple", "Banana", "Cherry"];
fruits.pop();
console.log(fruits);
```

Output:

```
["Apple", "Banana"]
```

C. shift()

Removes the **first element** from the array.

```
let fruits = ["Apple", "Banana", "Cherry"];
fruits.shift();
console.log(fruits);
```

Output:

```
["Banana", "Cherry"]
```

D. unshift()

Adds an element **at the beginning** of the array.

```
let fruits = ["Banana", "Cherry"];
fruits.unshift("Apple");
console.log(fruits);
```

Output:

```
["Apple", "Banana", "Cherry"]
```

Array Methods (map, filter, reduce, find, forEach)

A. map()

Creates a **new array** by applying a function to every element.

```
let numbers = [1, 2, 3, 4];
let squares = numbers.map(num => num * num);
console.log(squares);
```

Output:

```
[1, 4, 9, 16]
```

B. filter()

Creates a **new array** with elements that **pass a condition**.

```
let numbers = [10, 20, 30, 40];
let result = numbers.filter(num => num > 20);
console.log(result);
```

Output:

```
[30, 40]
```

C. reduce()

Reduces all elements to a **single value** (like sum, average, etc.)

```
let numbers = [10, 20, 30];
let sum = numbers.reduce((acc, num) => acc + num, 0);
console.log(sum);
```

Output:

60

Strings in JavaScript

A **string** is a sequence of characters enclosed in **single quotes** (`' '`), **double quotes** (`" "`), or **backticks** (```).

Example:

```
let name = "Neeraj";
let greeting = 'Hello';
let message = `Welcome, ${name}!`;
```

String Length

The `.length` property returns the number of characters in a string.

```
let text = "JavaScript";
console.log(text.length); // 10
```

Template Literals

- Introduced in ES6.
- Enclosed in **backticks** (```).
- Allow **multi-line strings** and **variable interpolation** using `${}`.

Example:

```
let firstName = "Neeraj";
let age = 23;

let info = `My name is ${firstName}.
I am ${age} years old.`;

console.log(info);
```

Output:

```
My name is Neeraj.
I am 23 years old.
```

String Methods (toUpperCase, toLowerCase, trim)

JavaScript provides several **built-in methods** to work with strings.

1. toUpperCase()

Converts all letters of a string to **uppercase**.

```
let name = "neeraj";
console.log(name.toUpperCase());
// Output: "NEERAJ"
```

2. toLowerCase()

Converts all letters of a string to **lowercase**.

```
let country = "INDIA";
console.log(country.toLowerCase());
// Output: "india"
```

3. trim()

Removes **extra spaces** from the **start and end** of a string.

```
let text = "  Hello JavaScript!  ";
console.log(text.trim());
// Output: "Hello JavaScript!"
```

Other Important String Methods

1. split()

Splits a string into an **array** based on a separator.

```
let fruits = "apple,banana,mango";
let arr = fruits.split(",");
console.log(arr);
// Output: ["apple", "banana", "mango"]
```

2. replace()

Replaces a part of a string with another value.

```
let text = "I like Java.";
let newText = text.replace("Java", "JavaScript");
console.log(newText);
// Output: "I like JavaScript."
```

3. substring(start, end)

Extracts a portion of the string between **start** and **end** indexes.

```
let word = "JavaScript";
console.log(word.substring(0, 4));
// Output: "Java"
```

4. includes()

Checks if a string **contains** a given substring.
Returns **true** or **false**.

```
let sentence = "Learning JavaScript is fun!";  
console.log(sentence.includes("JavaScript"));  
// Output: true
```

5. startsWith()

Checks if a string **starts with** a specific substring.

```
let text = "Hello World!";  
console.log(text.startsWith("Hello"));  
// Output: true
```

6. endsWith()

Checks if a string **ends with** a specific substring.

```
let text = "Hello World!";  
console.log(text.endsWith("World!"));  
// Output: true
```

Objects in JavaScript

What is an Object?

An **object** is a collection of **key-value pairs**.
Each key is called a **property**, and the value can be **any data type** (number, string, array, function, etc.).

1. Object Literal Syntax

```
let person = {  
  name: "Neeraj",  
  age: 23,  
  city: "Lucknow"  
};
```

Accessing Object Properties

```
console.log(person.name);    // Dot notation → "Neeraj"  
console.log(person["city"]); // Bracket notation → "Lucknow"
```

2. Adding & Deleting Properties

```
person.country = "India";    // Add new property  
delete person.age;           // Delete a property  
  
console.log(person);
```

3. Object Methods

A **method** is a function defined inside an object.

```
let student = {
  name: "Amit",
  marks: 90,
  greet: function() {
    console.log("Hello, my name is " + this.name);
  }
};

student.greet(); // Output: Hello, my name is Amit
```

Object Utilities

JavaScript provides built-in utility methods for working with objects.

1. Object.keys(obj)

Returns an **array of property names (keys)**.

```
let person = { name: "Neeraj", age: 23, city: "Lucknow" };
console.log(Object.keys(person));
// Output: ["name", "age", "city"]
```

2. Object.values(obj)

Returns an **array of property values**.

```
console.log(Object.values(person));
// Output: ["Neeraj", 23, "Lucknow"]
```

3. Object.entries(obj)

Returns an **array of [key, value] pairs**.

```
console.log(Object.entries(person));
// Output: [["name", "Neeraj"], ["age", 23], ["city", "Lucknow"]]
```

Looping through Object using Object.entries()

```
for (let [key, value] of Object.entries(person)) {
  console.log(`${key}: ${value}`);
}
```

Output:

```
name: Neeraj
age: 23
city: Lucknow
```

Object Destructuring

Destructuring allows you to **extract properties** from an object into **variables**.

Example 1: Basic Destructuring

```
let user = { name: "Neeraj", age: 23, city: "Lucknow" };

let { name, age } = user;

console.log(name); // "Neeraj"
console.log(age);  // 23
```

Example 2: Renaming Variables

```
let { name: fullName, city: hometown } = user;
console.log(fullName); // "Neeraj"
console.log(hometown); // "Lucknow"
```

Example 3: Default Values

If a property does not exist, you can assign a default value.

```
let { name, country = "India" } = user;
console.log(country); // "India"
```

Timers (setTimeout, setInterval)

Timers are used to **run code after a delay** or **repeatedly** after intervals.

setTimeout()

Executes a function **once** after a specified delay (in milliseconds).

Example:

```
setTimeout(function() {
  console.log("This message appears after 3 seconds");
}, 3000);
```

setInterval()

Executes a function **repeatedly** after a specified interval.

Example:

```
let count = 1;

let timer = setInterval(function() {
  console.log("Count:", count);
  count++;

  if (count > 5) {
    clearInterval(timer); // stop after 5 times
  }
}, 1000);
```

```
    }  
  }, 1000);
```

clearTimeout()

Cancels a timer set with `setTimeout()`.

```
let timeout = setTimeout(() => console.log("Will not run"), 5000);  
clearTimeout(timeout);
```

Popup Boxes (alert, prompt, confirm)

JavaScript provides **three popup boxes** for simple user interaction.

alert()

Displays a message box with an **OK** button.

```
alert("Welcome to JavaScript!");
```

prompt()

Displays a box asking for **user input**.
You can store the entered value in a variable.

```
let name = prompt("Enter your name:");  
alert("Hello, " + name + "!");
```

confirm()

Displays a box asking for **Yes/No confirmation**.
Returns `true` if OK is clicked, otherwise `false`.

```
let result = confirm("Are you sure you want to exit?");  
if (result) {  
    alert("Exiting...");  
} else {  
    alert("Cancelled!");  
}
```

Summary

Concept	Description
BOM	Interacts with browser (window, navigator, history, location)
Timers	Delay or repeat actions (<code>setTimeout</code> , <code>setInterval</code>)
Popups	User messages or confirmations (<code>alert</code> , <code>prompt</code> , <code>confirm</code>)

Escape Sequence Characters in JavaScript

Escape sequences are special characters used inside strings when you want to represent something that cannot be typed directly.

1. \n – New Line

Starts a new line.

```
console.log("Hello\nWorld");
```

2. \t – Tab

Inserts a horizontal tab space.

```
console.log("Name:\tNeeraj");
```

3. \\ – Backslash

Prints a real backslash.

```
console.log("This is a backslash: \\");
```

4. \' – Single Quote

Used to print a single quote inside a string.

```
console.log('It\'s a good day');
```

5. \" – Double Quote

Used to print double quotes inside a string.

```
console.log("He said \"Hello\"");
```

Destructuring (Array & Object)

Destructuring allows you to **extract values** from arrays or objects easily.

Array Destructuring

```
let numbers = [10, 20, 30];
```

```
let [a, b, c] = numbers;
```

```
console.log(a); // 10
```

```
console.log(b); // 20
```

```
console.log(c); // 30
```

Skippping values:

```
let [x, , z] = [1, 2, 3];
```

```
console.log(x, z); // 1 3
```

With default values:

```
let [p, q = 5] = [10];  
console.log(p, q); // 10 5
```

Spread & Rest Operators (...)

Both use ... but work **differently** depending on the context.

Spread Operator

Used to **expand** arrays or objects.

Example (Array):

```
let arr1 = [1, 2, 3];  
let arr2 = [...arr1, 4, 5];  
console.log(arr2); // [1, 2, 3, 4, 5]
```

Example (Object):

```
let obj1 = { name: "Neeraj", age: 23 };  
let obj2 = { ...obj1, city: "Lucknow" };  
console.log(obj2);  
// { name: "Neeraj", age: 23, city: "Lucknow" }
```

Rest Operator

Used to **collect remaining values** into an array or object.

Example (Array):

```
let [a, b, ...rest] = [10, 20, 30, 40, 50];  
console.log(a); // 10  
console.log(b); // 20  
console.log(rest); // [30, 40, 50]
```

Example (Function Parameters):

```
function sum(...numbers) {  
  return numbers.reduce((total, num) => total + num, 0);  
}  
console.log(sum(1, 2, 3, 4)); // 10
```

Console Object (console)

The **console object** is used to display messages in the browser's console (mainly for debugging).

Common Console Methods

a) `console.log()`

Prints general messages or values.

```
console.log("Hello JavaScript");
```

b) `console.error()`

Shows an error message in red.

```
console.error("Something went wrong!");
```

c) `console.warn()`

Displays a warning message in yellow.

```
console.warn("This is a warning");
```

d) `console.table()`

Displays data in a table form.

```
console.table([1, 2, 3]);
```

e) `console.clear()`

Clears the console.

```
console.clear();
```

f) `console.info()`

Shows informational messages.

```
console.info("This is some useful info.");
```

Quick Summary Table

Function	What it does	Return Value
<code>alert()</code>	Shows a message popup	No return (undefined)
<code>prompt()</code>	Takes user input	String or null
<code>confirm()</code>	Asks Yes/No	true / false
<code>console.log()</code>	Prints message	void
<code>console.error()</code>	Prints error	void
<code>console.warn()</code>	Prints warning	void

Function	What it does	Return Value
<code>console.table()</code>	Prints table	void

DOM (Document Object Model)

DOM represents the **HTML document** as a tree of objects (nodes).
Using DOM, we can **access, modify, add, or delete HTML elements**.

DOM = HTML + CSS as Objects

The browser converts your HTML into a JavaScript object called the **document**.

Common DOM Operations

a) Selecting Elements

```
document.getElementById("title");
document.querySelector(".box");
document.getElementsByClassName("item");
```

b) Changing Content

```
document.getElementById("title").innerHTML = "New Title";
```

c) Changing Styles

```
document.querySelector("h1").style.color = "red";
```

d) Creating New Elements

```
let btn = document.createElement("button");
btn.innerText = "Click Me";
document.body.appendChild(btn);
```

e) Event Handling

```
document.getElementById("btn").addEventListener("click", function() {
    alert("Button clicked!");
});
```

Clear Difference – Window vs BOM vs DOM

Feature	Window	BOM	DOM
Meaning	Main global browser object	Browser features	HTML document
Contains	BOM + DOM	location, history, navigator	HTML elements
Purpose	Control browser tab	Control browser features	Manipulate webpage
Example	<code>window.alert()</code>	<code>window.location.href</code>	<code>document.getElementById()</code>

In Simple Words

- **Window** → The boss (main container of everything)
- **BOM** → Browser features (URL, history, screen, alert, prompt)
- **DOM** → HTML elements (document, tags, text, attributes)

JavaScript DOM Nodes

In the DOM, everything inside an HTML document is represented as a **node**.

Types of DOM Nodes

1. **Element Node**
2. **Text Node**
3. **Comment Node**

Element Node

Represents an HTML element.

Example:

```
<p>Hello</p>
```

Here `<p>` is an **element node**.

JavaScript Example:

```
let para = document.querySelector("p");  
console.log(para.nodeType); // 1 (Element node)
```

2. Text Node

Represents the **text inside an element**.

Example:

```
<p>Hello</p>
```

The word **"Hello"** is a **text node**.

JavaScript Example:

```
let text = document.querySelector("p").firstChild;  
console.log(text.nodeType); // 3 (Text node)
```

3. Comment Node

Represents an HTML comment.

Example:

```
<!-- This is a comment -->
```

JavaScript Example:

```
let comment = document.body.childNodes[1];  
console.log(comment.nodeType); // 8 (Comment node)
```

Walking the DOM

“Walking the DOM” means moving through nodes like:

- parent → child
- child → parent
- element → next sibling
- element → previous sibling
- accessing descendants

4. Parent Node

The element **above** another node.

Example:

```
let child = document.querySelector("p");  
console.log(child.parentNode); // returns the parent element
```

5. Child Nodes

Nodes that are **directly inside** a parent.

Properties:

- `element.childNodes` → returns all child nodes (including text + comment)
- `element.children` → only element nodes

Example:

```
let list = document.querySelector("ul");  
console.log(list.children); // only <li> elements  
console.log(list.childNodes); // includes text nodes also
```

6. First Child & Last Child

firstChild

Returns the first node (can be a text node).

firstElementChild

Returns the first **element** node only.

Example:

```
let div = document.querySelector("div");
console.log(div.firstChild);           // maybe text node
console.log(div.firstElementChild);    // always element
```

lastChild

Returns the last node.

lastElementChild

Returns the last element node.

7. Sibling Nodes

Nodes that share the same parent.

Useful Properties:

- `nextSibling` (may return text node)
- `previousSibling`
- `nextElementSibling`
- `previousElementSibling`

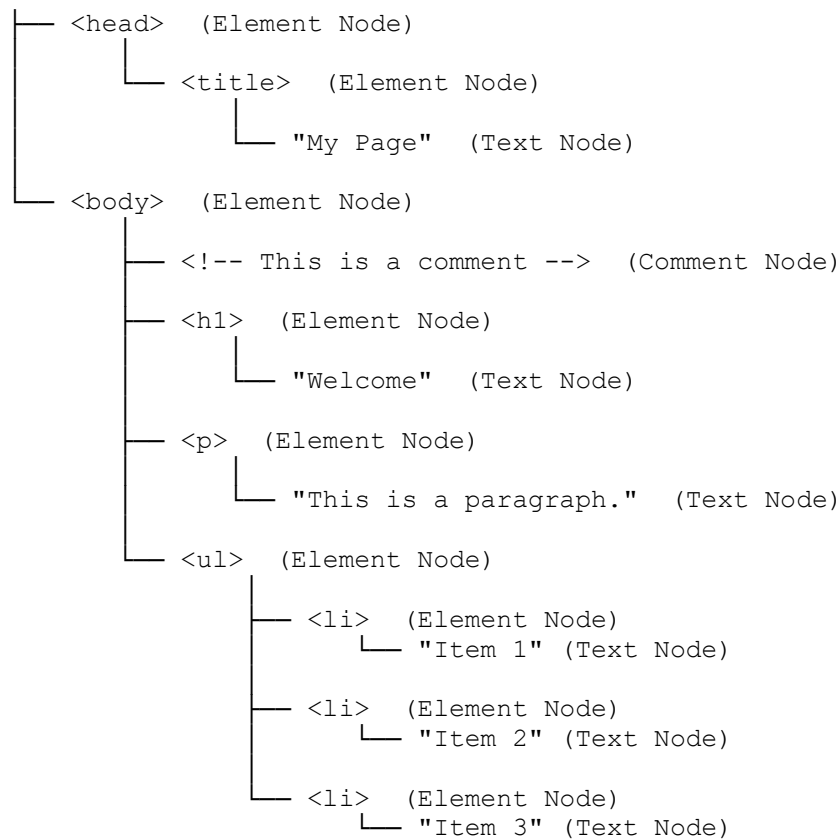
Example:

```
let item = document.querySelector("li");
console.log(item.nextElementSibling);
console.log(item.previousElementSibling);
```

Short Summary for Quick Revision

- **Element Node** → `<p>`, `<div>`, `<h1>`
- **Text Node** → Text inside elements
- **Comment Node** → `<!-- comment -->`
- **parentNode** → parent element
- **childNodes** → all children including text
- **children** → only element children
- **firstChild** / **lastChild** → first/last node
- **firstElementChild** / **lastElementChild** → first/last element
- **nextSibling** / **previousSibling** → may include text nodes
- **nextElementSibling** / **previousElementSibling** → only elements
- **descendants** → all nested nodes inside an element

Document
├── `<html>` (Element Node)
│
└──



Element-Only Navigation

These properties help you move between **elements only** (ignores text nodes, comments).

a) parentElement

Returns the parent **element**.

```
element.parentElement
```

b) children

Returns all **element children** (not text nodes).

```
let list = document.querySelector("ul");
console.log(list.children);
```

c) firstElementChild

Returns the first **element** child.

```
element.firstElementChild
```

d) lastElementChild

Returns the last **element** child.

```
element.lastElementChild
```

e) nextElementSibling

Returns the next **element sibling**.

```
element.nextElementSibling
```

f) previousElementSibling

Returns the previous **element sibling**.

```
element.previousElementSibling
```

DOM Events (Basics)

Events are actions performed by the user or browser. Common events:

Event	Meaning
click	User clicks
dblclick	Double-click
input	User types in input
change	Input value changed
mouseover	Mouse on element
mouseout	Mouse leaves element
keydown	Key pressed
submit	Form submitted
load	Page loaded

Add event listener:

```
element.addEventListener("click", function () {  
    console.log("Clicked!");  
});
```

console.dir()

Used to inspect an element as a **JavaScript object**.

```
console.dir(document.body);
```

Shows:

- properties
- methods
- events
- attributes

Used more than `console.log()` for DOM debugging.

tagName and nodeName

tagName

- Works only for **element nodes**
- Always returns **UPPERCASE tag name**

```
let p = document.querySelector("p");
console.log(p.tagName); // "P"
```

nodeName

- Works for **all node types**
- Element node → tag name
- Text node → `"#text"`
- Comment node → `"#comment"`

```
console.log(p.firstChild.nodeName); // "#text"
```

innerHTML

Returns or sets the **HTML inside** an element.

```
div.innerHTML = "<b>Hello</b>";
```

Includes tags + text.

outerHTML

Returns or replaces the **entire element including itself**.

```
div.outerHTML = "<p>Replaced Div</p>";
```

textContent

Returns only **plain text**, no HTML tags.

```
div.textContent = "Hello World";
```

hidden Property

- A boolean that hides an element when set to `true`.

```
element.hidden = true; // hides element
```

```
element.hidden = false; // shows element
```

Same as adding `hidden` attribute in HTML.

DOM Insertion Methods

1) append()

Adds element **at the end** inside parent.

```
parent.append(child);
```

2) prepend()

Adds element **at the beginning** inside parent.

```
parent.prepend(child);
```

3) before()

Adds element **before** the current element.

```
element.before(newNode);
```

4) after()

Adds element **after** the current element.

```
element.after(newNode);
```

5) replaceWith()

Replaces an element.

```
element.replaceWith(newNode);
```

insertAdjacentHTML / Element / Text

These allow inserting content in 4 positions:

Position	Meaning
"beforebegin"	Before the element
"afterbegin"	Inside, at the start
"beforeend"	Inside, at the end
"afterend"	After the element

Example:


```
element.insertAdjacentHTML("beforebegin", "<p>Before</p>");
element.insertAdjacentHTML("afterbegin", "<p>Start</p>");
element.insertAdjacentHTML("beforeend", "<p>End</p>");
element.insertAdjacentHTML("afterend", "<p>After</p>");
```

Node Removal Methods

1) remove()

Remove element itself.

```
element.remove();
```

2) removeChild()

Remove a specific child.

```
parent.removeChild(child);
```

className

Gets or sets the **full class string**.

```
div.className = "box highlight";
```

Replaces all classes.

Browser Events

Events represent actions performed in the browser.

Common Event Types

Event	Meaning
click	Mouse click
dblclick	Double click
mouseover	Mouse enters element
mouseout	Mouse leaves element
keydown	Key is pressed
keyup	Key released
input	User typing in input field
submit	Form submission
scroll	User scrolls page
load	Page has loaded
contextmenu	Right-click

Inline vs Modern Event Handling

Inline (Old way)

```
<button onclick="hello()">Click</button>
```

Modern way (Recommended)

```
button.addEventListener("click", hello);
```

Event Flow

Events move in 3 phases:

1. **Capturing Phase** (top → target)
2. **Target Phase**
3. **Bubbling Phase** (target → top)

Example to stop bubbling

```
event.stopPropagation();
```

Keyboard Event Example

```
document.addEventListener("keydown", (e) => {  
  console.log("Key:", e.key);  
});
```

Form Submit Event

```
form.addEventListener("submit", (e) => {  
  e.preventDefault(); // stops form from reloading  
  console.log("Form Submitted");  
});
```

10. Mouse Position Example

```
document.addEventListener("mousemove", (e) => {  
  console.log(e.clientX, e.clientY);  
});
```

Callback Functions

A **callback** is a function that is passed as an argument to another function and executed later.

Example

```
function greet(name, callback) {  
  console.log("Hello " + name);
```

```
    callback();  
  }  
  
  greet("Neeraj", () => {  
    console.log("Welcome!");  
  });  
};
```

Where callbacks are used?

- setTimeout()
- Event listeners
- API requests (old style)
- File reading

Callback Hell (Pyramid of Doom)

When too many callbacks are nested inside each other, code becomes:

- Hard to read
- Hard to debug
- Hard to maintain

Example (Callback Hell)

```
setTimeout(() => {  
  console.log("1");  
  
  setTimeout(() => {  
    console.log("2");  
  
    setTimeout(() => {  
      console.log("3");  
  
      }, 1000);  
    }, 1000);  
  }, 1000);
```

This deep pyramid structure is called the **Pyramid of Doom**.

Promises in JavaScript

A **Promise** represents a value that may be available now, later, or never.

States of a Promise

1. **Pending**
2. **Fulfilled (resolved)**
3. **Rejected**

Creating a Promise

```
let promise = new Promise((resolve, reject) => {
```

```
let success = true;

if (success) resolve("Done");
else reject("Error");
});
```

.then() and .catch()

Used to handle results of a promise.

.then() → for success

```
promise.then(result => {
  console.log("Success:", result);
});
```

.catch() → for errors

```
promise.catch(error => {
  console.log("Error:", error);
});
```

Promise Chaining

You can chain multiple `.then()` calls in sequence.

Example

```
new Promise(resolve => resolve(1))
  .then(num => {
    console.log(num);
    return num + 1;
  })
  .then(num => {
    console.log(num);
    return num + 1;
  })
  .then(num => console.log(num));
```

Each `.then()` returns a new **Promise**.

Async / Await & Error Handling

async Function

The **async** keyword is used to create a function that always returns a **Promise**.

Syntax

```
async function myFunc() {
  return "Hello";
}
```

This automatically becomes:

```
Promise.resolve("Hello")
```

await Keyword

await pauses the execution of an async function until a **Promise** settles.

Syntax

```
let result = await promise;
```

Example

```
function delay(ms) {  
  return new Promise(resolve => setTimeout(resolve, ms));  
}  
  
async function start() {  
  console.log("Waiting...");  
  await delay(2000);  
  console.log("Done!");  
}  
  
start();
```

Note: await works **only inside async functions**.

async + await Example

```
async function getUser() {  
  let response = await fetch("https://api.example.com/user");  
  let data = await response.json();  
  console.log(data);  
}  
  
getUser();
```

Error Handling with try...catch

Errors generated inside async functions can be handled using **try-catch**.

Example

```
async function fetchData() {  
  try {  
    let res = await fetch("https://wrong-url.com/api");  
    let data = await res.json();  
    console.log(data);  
  } catch (error) {  
    console.log("Error Occurred:", error);  
  }  
}
```

```
fetchData();
```

finally Block

The **finally** block always runs:

- whether try succeeds
- or catch catches error

Used for cleanup code.

Example

```
async function processData() {
  try {
    let response = await fetch("/api");
    let data = await response.json();
    console.log(data);

  } catch (err) {
    console.log("Error:", err);
  } finally {
    console.log("Operation Finished");
  }
}
```

Throwing Custom Errors

You can throw your own meaningful error messages.

Syntax

```
throw new Error("Custom error message");
```

Example

```
function divide(a, b) {
  if (b === 0) {
    throw new Error("Division by zero is not allowed");
  }
  return a / b;
}
```

Handling the custom error:

```
try {
  divide(10, 0);
} catch (err) {
  console.log("Caught:", err.message);
}
```

Throwing Errors Inside async Functions

```

async function getData() {
  try {
    let response = await fetch("url");

    if (!response.ok) {
      throw new Error("Failed to fetch: " + response.status);
    }

    let data = await response.json();
    return data;

  } catch (err) {
    console.log("Error:", err);
  }
}

```

Using await with Promise Chains

```

async function run() {
  let step1 = await Promise.resolve("A");
  let step2 = await Promise.resolve("B");
  let step3 = await Promise.resolve("C");

  console.log(step1, step2, step3);
}
run();

```

Re-throwing Errors

You can catch an error and throw it again.

```

async function loadData() {
  try {
    let data = await fetch("/api");
    return data;

  } catch (err) {
    console.log("Handled locally, sending up...");
    throw err;    // re-throw
  }
}

```

Summary

Concept	Meaning
async	Makes a function return a Promise
await	Waits for a Promise result
try	Code that may fail
catch	Catches errors
finally	Runs always
throw	Creates custom errors

What is OOP? (Object-Oriented Programming)

OOP is a programming style where everything is represented as **objects**.

Objects contain:

- **Properties** → variables (data)
- **Methods** → functions (behavior)

Goals of OOP

- Organize code into reusable structures
- Reduce duplication
- Make code easy to maintain
- Provide real-world modeling

Pillars of OOP

1. **Encapsulation** – bundling data & methods inside an object
2. **Inheritance** – one class can use properties/methods of another
3. **Polymorphism** – same method behaves differently
4. **Abstraction** – hide unnecessary details

Objects in JavaScript

In JavaScript, objects are key–value pairs.

Example

```
let user = {  
  name: "Neeraj",  
  age: 23,  
  greet() {  
    console.log("Hello!");  
  }  
};
```

What is a Constructor?

A **constructor** is a special function used to create and initialize objects.

In JavaScript, constructors are usually written using:

- Function constructor (old)
- `class` constructor (modern, ES6)

Constructor Function (Old Style)

Syntax

```
function Person(name, age) {  
  this.name = name;  
  this.age = age;  
}
```



```
this.greet = function() {  
    console.log("Hello " + this.name);  
};  
}
```

Creating objects:

```
let p1 = new Person("Aman", 22);  
let p2 = new Person("Rahul", 25);  
  
p1.greet();
```

`new` keyword creates a new object and binds `this` to it.

Constructor in ES6 Classes (Modern)

Modern and recommended way.

Syntax

```
class Car {  
    constructor(model, color) {  
        this.model = model;  
        this.color = color;  
    }  
  
    start() {  
        console.log(this.model + " Started");  
    }  
}
```

Creating objects:

```
let c1 = new Car("BMW", "Black");  
c1.start();
```

Points About Constructors

- Only one constructor per class
- Automatically called when object is created
- Used to set initial values
- No return statement required
- If you don't write a constructor, JS adds an **empty constructor** automatically

Default Constructor

If no constructor is defined:

```
class Student {}
```

JS internally creates:

```
constructor() {}
```

Example: Class With Constructor

```
class Student {
  constructor(name, city) {
    this.name = name;
    this.city = city;
  }

  info() {
    console.log(this.name + " from " + this.city);
  }
}

let s1 = new Student("Neeraj", "Lucknow");
s1.info();
```

Real-life Example

```
class BankAccount {
  constructor(owner, balance) {
    this.owner = owner;
    this.balance = balance;
  }

  deposit(amount) {
    this.balance += amount;
  }
}

let a1 = new BankAccount("Neeraj", 5000);
console.log(a1.balance);
```

What is Inheritance?

Inheritance allows one class to **use the properties and methods of another class**.

- The base class is called **Parent class / Super class**
- The derived class is called **Child class / Subclass**

It helps in:

- Reusability
- Reducing code duplication
- Creating hierarchical relationships

Use the `extends` keyword.

Example

```
class Animal {
  eat() {
```

```

        console.log("Animal eats food");
    }
}

class Dog extends Animal {
    bark() {
        console.log("Dog barks");
    }
}

let d = new Dog();
d.eat();    // from parent class
d.bark();  // from child class

```

Constructor in Inheritance

If both classes have constructors, then child class must use **super()** inside the constructor.

Example

```

class Parent {
    constructor(name) {
        this.name = name;
    }
}

class Child extends Parent {
    constructor(name, age) {
        super(name);    // calling Parent constructor
        this.age = age;
    }
}

let c = new Child("Neeraj", 23);

```

What is super Keyword?

super is used to:

A. Call parent class constructor

```
super(arguments)
```

B. Access parent class methods

```
super.methodName()
```

Using super() in Constructor

Example

```

class Vehicle {
    constructor(company) {
        this.company = company;
    }
}

```

```

class Car extends Vehicle {
  constructor(company, model) {
    super(company); // parent constructor
    this.model = model;
  }
}

let obj = new Car("Toyota", "Fortuner");
console.log(obj.company);

```

Local Scope & Global Scope

Global Scope

A variable that is declared **outside any function or block** is global. It can be accessed **anywhere** in the program.

Example

```

let globalVar = "I am global";

function test() {
  console.log(globalVar); // Accessible inside the function
}

test();
console.log(globalVar); // Also accessible here

```

Local Scope

A variable declared **inside a function or block { }** is local. It can be accessed **only inside that block/function**.

Example

```

function demo() {
  let localVar = "I am local";
  console.log(localVar); // Accessible
}

demo();
console.log(localVar); //Error (not accessible outside)

```

Block Scope (with let/const)

let and const are block-scoped.

```

if (true) {
  let x = 10;
  const y = 20;
}

console.log(x); //Error

```

```
console.log(y); //Error
```

Fetch API (Modern way to make HTTP requests)

The **Fetch API** is used to send requests (GET, POST, PUT, DELETE) to a server. It returns a **Promise**.

Example: GET Request

```
fetch("https://api.example.com/data")
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.log("Error:", error));
```

Example: POST Request

```
fetch("https://api.example.com/users", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    name: "Neeraj",
    age: 20
  })
})
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.log(err));
```

Local Storage (Stored permanently until cleared)

- Stores data **without expiration date**
- Data stays even after closing the browser
- Max storage ~10MB
- Stores only **strings**

Save Data

```
localStorage.setItem("name", "Neeraj");
```

Get Data

```
let name = localStorage.getItem("name");
console.log(name);
```

Remove Data

```
localStorage.removeItem("name");
```

Clear All

```
localStorage.clear();
```

Store Objects (JSON)

```
let user = { name: "Neeraj", age: 20 };

localStorage.setItem("user", JSON.stringify(user));
```

Retrieve:

```
let data = JSON.parse(localStorage.getItem("user"));
console.log(data.name);
```

Cookies in JavaScript

- Small data stored in browser
- Max size around **4 KB**
- Can have **expiration time**
- Sent automatically with **HTTP requests**

Set Cookie

```
document.cookie = "username=Neeraj; expires=Wed, 21 Aug 2025 12:00:00 UTC; path="/;
```

Read Cookies

```
console.log(document.cookie);
```

Delete Cookie

```
document.cookie = "username=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path="/;
```

Difference Between LocalStorage, SessionStorage & Cookies

Feature	Local Storage	Session Storage	Cookies
Storage Size	~10MB	~5MB	~4KB
Expires	Never	Close tab/browser	Set by server
Sent with HTTP requests	No	No	Yes
Use Case	Long-term data	Temporary data	Login, tracking

Short Summary

- **Fetch API** → For calling APIs
- **Local Storage** → Permanent storage
- **Session Storage** → Temporary tab-based storage
- **Cookies** → Small data + backend usage